



UNIVERSITY of
DEBRECEN



Building a Rule-Based Line Follower Robot: Design, Testing, and Performance Insights

Animon, Azar
Department of Mechatronics
The University of Debrecen
Debrecen, Hungary
azar.ani@mailbox.hu

Pani, David
Department of Mechatronics
The University of Debrecen
Debrecen, Hungary

Osakwe, Nmesoma
Department of Mechatronics
The University of Debrecen
Debrecen, Hungary

Hiba, Malak
Department of Mechatronics
The University of Debrecen
Debrecen, Hungary

John Stephen Barka
Department of Mechatronics
The University of Debrecen
Debrecen, Hungary

Abstract—

This paper details the design, development, and evaluation of a rule-based line follower robot built using infrared (IR) sensors, an Arduino Uno, a dual-channel L298N motor driver, and DC motors. The robot utilizes a thresholding approach to interpret line patterns and make directional decisions in real-time. While traditional PID (Proportional–Integral–Derivative) controllers are widely used in similar applications for smoother and adaptive control, this project opts for a rule-based logic system due to hardware limitations and simplicity of implementation. The bot's performance was tested across various track scenarios, including sharp curves, intersections, and path losses. In addition to behavioral analysis, lap timing data was recorded to assess consistency, response sharpness, and recovery time from disruptions. These performance metrics were used to quantify the effectiveness of the rule-based approach. The paper also compares rule-based logic with PID control and discusses potential enhancements, such as incorporating sensor fusion, feedback mechanisms, and adaptive thresholds, to improve long-term accuracy and adaptability. References to current research on control algorithms such as PID, fuzzy logic, and decision-tree-based line following are also provided to position our approach in the context of existing work

I. INTRODUCTION

Autonomous line-following robots represent a foundational application of robotics and control systems, offering practical insight into sensor integration, decision-making algorithms, and real-time feedback control. These robots are widely employed across multiple industries including manufacturing, warehouse logistics, surveillance, construction, and mining due to their ability to automate repetitive transportation tasks with high accuracy and minimal human intervention [1][2]. In educational contexts, they are equally invaluable, serving as entry points for learning about embedded systems, algorithm design, and electronics prototyping. With the advancement of intelligent automation and Industry 4.0, the relevance of mobile autonomous systems like line-followers has grown significantly. Modern industrial systems increasingly

depend on agile, low-cost solutions capable of navigating predefined routes or adapting to dynamic environments. This demand has begun for diverse research into line-following robot control strategies—ranging from classical rule-based systems to more sophisticated PID controllers, fuzzy logic, and even neural network-based models that leverage machine learning to adaptively learn and optimize path-tracking behavior [4]. In the academia space and low-cost prototyping, rule-based control systems are still preferred for their simplicity, ease of implementation, and suitability for resource-constrained environments. While PID control offers superior smoothness and adaptability in high-performance systems, it often demands precise calibration, robust hardware, and high-quality sensor feedback—factors that may not be feasible in all student projects or educational competitions. Rule-based logic provides immediate, deterministic behavior suitable for deterministic responses, making it particularly attractive in systems where design constraints outweigh flexibility needs.

This paper presents the design, development, and performance evaluation of a low-cost rule-based line-following robot, constructed using readily available components including five analog infrared (IR) sensors, an Arduino Uno microcontroller, and a dual-channel L298N motor driver. The robot was designed for a university-level internal competition and tested on a challenging custom-built track featuring curves, intersections, and obstacles. The control logic is based on binary thresholding of sensor data and discrete movement functions tailored to predefined sensor patterns. While many existing studies emphasize PID or fuzzy control for path tracking, our work explores the benefits and limitations of rule-based logic within a constrained hardware environment.

In addition to fulfilling its educational purpose, this project holds broader technological relevance. The knowledge and methods applied here reflect the same principles used in scaled industrial applications—where line-following robots manage goods, navigate structured environments, or serve as mobile inspection platforms.

By simplifying and contextualizing these complex systems for learning and prototyping, this work contributes to the democratization of robotics and enhances accessibility for students and hobbyists alike.

Moreover, the robot's development process, from circuitry to mechanical design and software implementation, mirrors real-world engineering workflows. Its evaluation using video-based kinematic analysis (via Kinovea) offers a quantitative approach to measuring performance, underscoring the increasing integration of data-driven techniques in control system design and validation. This multidisciplinary convergence reflects current trends in robotics research, where hardware, software, and analytics coalesce to produce intelligent, responsive machines. Thus, this paper documents a functional robotic system but also situates it within the broader landscape of control theory, embedded systems education, and robotics research and it highlights the practical viability of simple systems while paving the way for future iterations involving more complex logic and smarter decision-making capabilities.

II. METHODOLOGY

The design and implementation of the line-following robot were based on a compact and efficient integration of five analog infrared (IR) reflective sensors, an L298N motor driver, two DC motors, and an Arduino UNO microcontroller. The system was built on a lightweight chassis equipped with two wheels for mobility.

Five analog IR sensors were mounted at the front of the robot in a linear array to detect the position of a black line against a lighter background. Each sensor outputs a voltage proportional to the surface reflectivity, allowing the Arduino to determine the position of the line. These sensors were connected to the analog input pins A0 through A4 on the Arduino UNO. All sensors were powered via Arduino's 5V and GND pins, and their readings were used as the primary input for the line detection logic.

An Arduino UNO served as the central processing unit. It continuously read analog values from the IR sensors and interpreted them to identify the robot's position relative to the line. The Arduino was programmed using the Arduino IDE, allowing easy development and modification of the control algorithm. The logic was implemented in a rule-based manner to handle various sensor input combinations, enabling the robot to make real-time decisions such as turning left, right, or moving straight.

Choosing the appropriate control algorithm is central to a line-following robot's performance. Different algorithms offer varying degrees of accuracy, complexity, and responsiveness. In this section, we discuss three primary control strategies: Rule-Based Logic, PID Control, and Fuzzy Logic Control, with particular focus on PID due to its widespread application and theoretical foundation.

Rule-based logic is one of the most fundamental approaches used in robotics. It operates by mapping specific sensor inputs directly to predefined motor actions. For instance, if the central sensor detects the line while the others do not, the robot proceeds straight. If the leftmost sensors detect the line, the robot turns left, and similarly for the right side. This approach is suitable for environments with minimal variation, such as simple, straight tracks or those with gradual curves [5]. The main advantage of the method lies in its simplicity. It is easy to implement microcontrollers like the Arduino, requiring minimal resources, and behaves in a predictable manner. However, it lacks adaptability and error correction capabilities. It struggles with

complex tracks or high-speed operation and can exhibit unstable behavior during sharp turns or sudden changes in the path.

In contrast, PID (Proportional-Integral-Derivative) control is a more complex feedback-based algorithm that dynamically adjusts the robot's output based on the difference between the current sensor reading and the desired trajectory/motion. The proportional term reacts to the current deviation, the integral term accounts for past cumulative errors, and the derivative term predicts future trends to dampen oscillations. PID offers a balanced solution between control performance and implementation complexity. It allows smoother turns, reduces overshoots, and improves line-following accuracy, especially at higher speeds. However, tuning the three parameters (K_p , K_i , and K_d) can be a challenge and often require trial-and-error or heuristic methods. Additionally, PID control is more sensitive to noise and requires additional processing compared to rule-based logic. A detailed overview of the development and applications of PID control is presented by Haowen et al. [1] and V. M. Alfaro et al. [3]. Beyond robotics, PID control is also applied in areas such as power systems, as demonstrated by Weibin et al. [6], further emphasizing its versatility.

An alternative to PID is Fuzzy Logic Control (FLC), which mimics human decision making by interpreting inputs in terms like 'slightly left' or 'strong right', rather than relying on fixed thresholds. This enables a more flexible and intuitive response to uncertain or noisy data [4]. Fuzzy logic is particularly robust under changing environmental conditions, such as variable lighting or inconsistent line quality. It provides smoother transitions between actions and can outperform traditional methods in complex or irregular track layouts. However, the development of a fuzzy logic controller involves designing membership functions and a comprehensive rule base, which can increase development time and complexity. While not as light as rule-based systems, fuzzy logic remains a strong control strategy for line-following applications when precision and adaptability are required.

The L298N motor driver module was used to control the speed and direction of two 6V DC motors (rated at 1000 RPM). The driver received directional signals through IN1–IN4 pins and speed control signals via PWM-enabled ENA and ENB pins from the Arduino. This allowed for both bidirectional movement and variable speed control. The L298N was powered by a 7.4V lithium-ion battery, which provided sufficient voltage to drive the motors after accounting for the internal voltage drop (~1.5V) of the motor driver.

The chassis provided a stable structural base for mounting all components, including the Arduino, motor driver, sensors, and battery. Two wheels, each driven by a DC motor, enabled the robot to maneuver based on the control signals. The arrangement minimized wiring clutter and facilitated straightforward debugging and maintenance.

All components shared a common ground to ensure electrical stability and prevent erratic behavior. During programming, the Arduino could be powered via USB, while the motors remained powered by the battery. This separation of power sources during development simplified code uploading and testing.

This implementation reflects a practical and cost-effective solution, allowing efficient line tracking through simple yet effective hardware-software integration [2].

III. CIRCUITRY

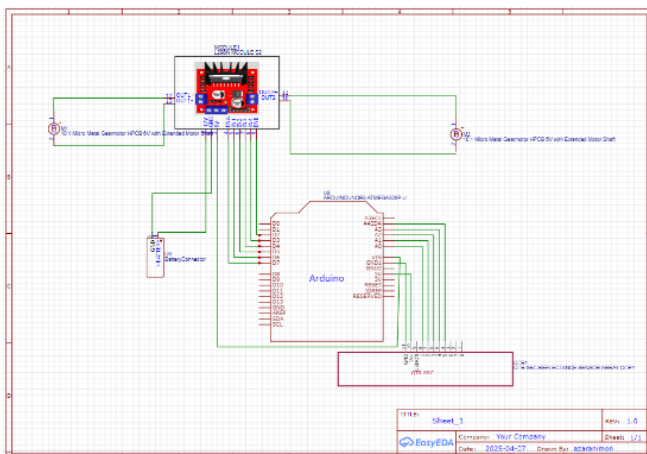
The circuit for the line-following robot was designed to integrate five analog IR sensors, an **L298N motor driver**, and an **Arduino UNO** in a compact and functional layout. Each **IR sensor** was connected to analog input pins A0 through A4 on the Arduino, enabling it to read reflectivity values essential for tracking the black line.

The **L298N motor driver** received directional and speed signals from the **Arduino** via digital pins and was powered using a 7.4V lithium-ion battery, which ensured consistent motor operation despite the internal voltage drop of approximately 1.5V in the driver [6].

The main connections in the circuit were as follows:
IR Sensors → Arduino A0-A4 (analog), 5V (VCC), and GND
Arduino → L298N IN1-IN4 (motor direction), ENA/ENB (PWM speed control)

Motors → Connected to L298N OUT1/OUT2 and OUT3/OUT4 terminals

Power Supply → 7.4V Li-ion battery connected to L298N VIN; Arduino optionally powered via USB during programming
All components shared a common ground to prevent electrical instability. The simplicity of this layout facilitated easy assembly and allowed focused troubleshooting. This design aligns with previous low-cost implementations of line-following robots in academic projects [2][5].



[Figure 2 (Finalised Circuit Mapping for LineFollower)]

IV. FINAL DESIGN

The final version of our line-following robot was built on a perforated prototyping board, which serves as both the base and the structure for mounting all components. This setup allows for easy adjustments and gives the bot a lightweight but sturdy frame.

At the front, a 5-channel infrared (IR) sensor array is mounted horizontally. Its placement is ideal for detecting the black line beneath the robot, allowing it to sense left, right, and center positions. Foam padding beneath the sensor module provides some shock absorption and stability, which helps reduce noise in readings. The sensor is connected via a ribbon cable, keeping the wiring tidy and reducing clutter.

The central component is an Arduino Uno board, which acts as the "brain" of the robot. It processes the signals from the sensors and sends commands to the motors. Placing it in the middle helps balance the weight and ensures easy access to pins for connections.

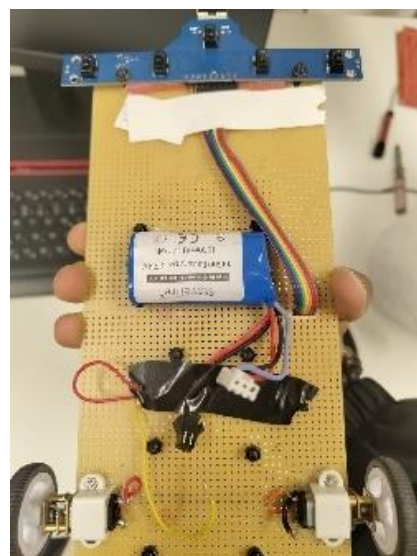
Power comes from a 7.4V lithium battery pack fixed near the center of the board, helping to maintain balance and avoid tipping. The battery is lightweight yet provides enough power for the motors and control electronics to run reliably.

At the back of the robot, two DC motors with wheels are mounted on each side. These provide the movement and turning ability. The motors are controlled by an L298N motor driver module, also located toward the rear. This motor driver allows the Arduino to control the speed and direction of the motors efficiently. The placement at the back keeps the wiring between the motors and driver short and direct.

This overall layout was chosen to keep the centre of gravity low and well-balanced, improving stability and control. Each component's placement contributes to the robot's performance—whether by improving line detection, enabling efficient motor control, or making the bot easier to assemble and maintain.



[Figure 3 (Top down view of Line Follower Bot)]



[Figure 4 (Underview of Line follower bot)]

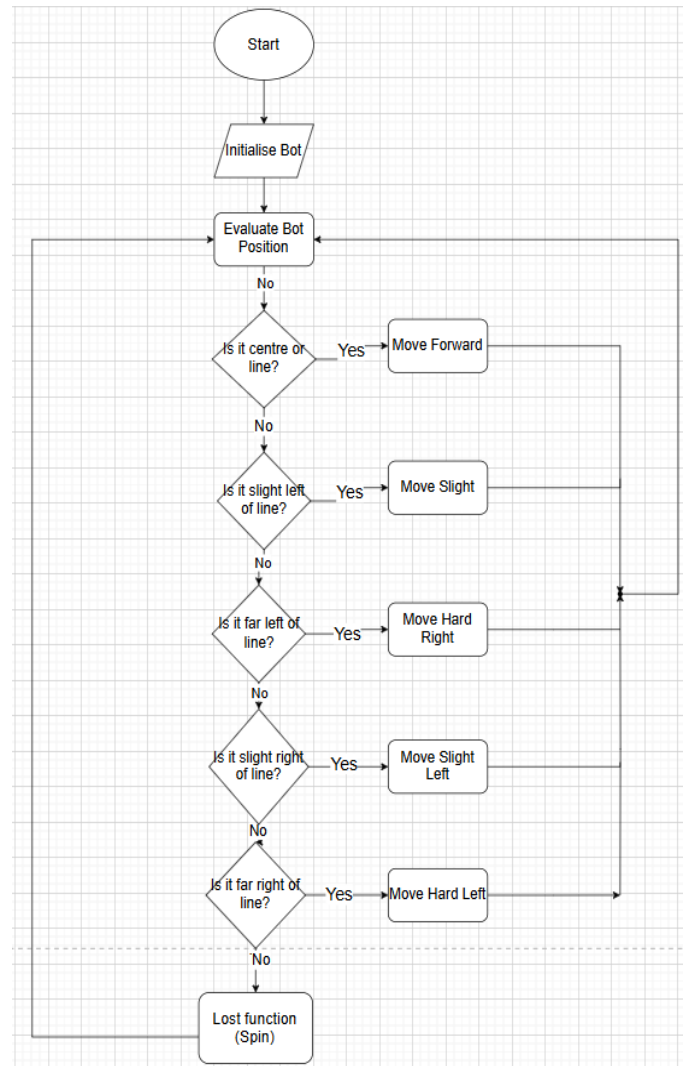
V. RULE BASED IMPLEMENTATION

The choice to implement a rule-based logic system for the line follower robot is primarily influenced by the physical constraints and operational behavior observed during development and testing. Although PID control is theoretically more accurate and widely adopted in line-following systems for its dynamic error correction and smoother path tracking [7], our implementation favors rule-based control for its predictability, simplicity, and resilience against hardware inconsistencies.

One of the primary challenges is the robot's weight distribution. Our design features a rear-heavy configuration due to the placement of the battery pack, motor driver (L298N), and Arduino Uno toward the back of the chassis. Additionally, the two driven wheels are also located at the rear, which leads to instability at the front end. This rear-load causes the front IR sensor array to lift slightly during acceleration or when navigating fast corners. The imbalance results in jerky motion and oversteer, especially during sharp turns or abrupt path corrections. Such dynamics make fine-grained proportional corrections, as used in PID control, ineffective in maintaining stable motion.

The IR sensor array spacing is another factor affecting performance. We use five analog reflective IR sensors arranged with approximately 15 mm spacing. This compact layout ensures high-resolution detection of narrow curves and subtle deviations, but it also amplifies sensor overlap and sensitivity to ambient lighting variations. Analog readings can fluctuate without optical isolation, making it difficult to compute a stable and meaningful error signal for PID computation. Instead, a binary thresholding approach provides a more reliable data stream by converting analog values into discrete ON/OFF states, which align well with the rule-based system.

After evaluating all this we initially designed our flowchart based on rule following structure and created a rough outline:



[Figure 8(The initial framework of the movement logic of the line follower)]

Sensor readings are fed to analog input pins A0 through A4 on the Arduino Uno. The microcontroller reads these values, applies thresholding logic to convert them into binary states, and stores the results in an array. Each unique binary pattern corresponds to a specific line position scenario, with logic inverted such that 1 indicates white (no line detected) and 0 indicates black (line detected):

- 11011** → Move Forward
- 10011** or **10111** → Turn Slight Right
- 01111** or **00111** → Turn Sharp Right
- 11101** or **11001** → Turn Slight Left
- 11100** or **11110** → Turn Sharp Left
- 11111** → Lost Line (Stop or recover based on last direction)

Each of these patterns maps directly to one of six custom movement functions defined in the control logic:

moveForward() – Drives both motors forward at equal speed.

turnLeft() – Slows the right motor and speeds up the left to gently veer right.

turnRight() – Slows the left motor and speeds up the right to veer left.

sharpLeft() – Pivots the robot sharply right by reversing the right motor and driving the left motor forward.

sharpRight() – Pivots sharply left by reversing the left motor and driving the right motor forward.

stopMotors() – Halts both motors, typically used when the line is lost.

Now to understand the mathematics behind this logic we represent the state of these sensors as a binary vector:

$$S = [s_1, s_2, s_3, s_4, s_5] \in \{0,1\}^5$$

Here, each element s_i of the vector corresponds to a single IR sensor:

- $s_i = 0$ means the sensor detects the line (dark surface),
- $s_i = 1$ means it sees the background (light surface).

This binary encoding gives us a total of $2^5 = 32$ possible combinations or sensor states, although in practice, only a subset of these are actually feasible when the robot is properly aligned with the line.

Once a valid sensor state S is observed, the robot determines the correct motor response using a function f , which maps sensor patterns to a discrete set of control actions:

$f: S \rightarrow A$, where $A = \{F, L, R, SL, SR, H\}$

We can say that each letter corresponds to a maneuver:

- **F** – Go Forward
- **L** – Slight Left Turn
- **R** – Slight Right Turn
- **SL** – Sharp Left Turn
- **SR** – Sharp Right Turn
- **H** – Halt (stop the motors)

Function	Manoeuvre	Line detection
$f([1,1,0,1,1])$	F	line is centred
$f([1,0,0,1,1])$	L	line is slightly left
$f([1,1,0,0,1])$	R	line is slightly right
$f([0,0,1,1,1])$	SR	line is sharply right
$f([1,1,1,0,0])$	SL	line is sharply left
$f([1,1,1,1,1])$	H	line lost or intersection

We also introduce a symmetry rule for mirrored actions:

$$f([s_1, s_2, s_3, s_4, s_5]) = \varphi(f([s_5, s_4, s_3, s_2, s_1]))$$

Each action $a \in A$ corresponds to a specific motor configuration: speeds and directions for the left and right motors. This is handled by a second function g :

$$g: A \rightarrow \mathbb{Z}^4$$

Function	Manoeuvre	Motor config	Line detection
$g(F)$	F	$(v, v, +1, +1)$	both motors forward
$g(L)$	L	$(v, \alpha v, +1, +1)$	slow right motor
$g(R)$	R	$(\alpha v, v, +1, +1)$	slow left motor
$g(SL)$	SR	$(v, v, +1, -1)$	rotate left
$g(SR)$	SL	$(v, v, -1, +1)$	rotate right
$g(H)$	H	$(0, 0, 0, 0)$	stop

The scalar factor α (typically 0.3 to 0.8) reduces speed for smooth turning.

Complete Control Loop: Composition of Logic

By chaining the two functions f and g , we define the control law:

$$\Gamma(S) = g(f(S))$$

At each control cycle:

- Read sensor state S_t
- Compute action $f(S)$
- Convert to motor signals $g(f(S))$
- Send commands to motors

This system acts as a memoryless finite-state machine, reacting to current input only.

Sensor patterns like all-black or all-white (e.g., $[0,0,0,0,0]$ or $[1,1,1,1,1]$) are ambiguous. To avoid erratic behavior, we use the last known state:

$$\Gamma(S_{\text{amb}}) = \Gamma(S_{t-1})$$

This serves as a basic memory mechanism to preserve direction.

Efficiency and Real-Time Performance

The logic is optimized for real-time systems with limited resources (e.g., Arduino). Only a small subset of sensor states is valid (typically 10–12), so we use a lookup table:

- Worst-case time: **$O(k)$**

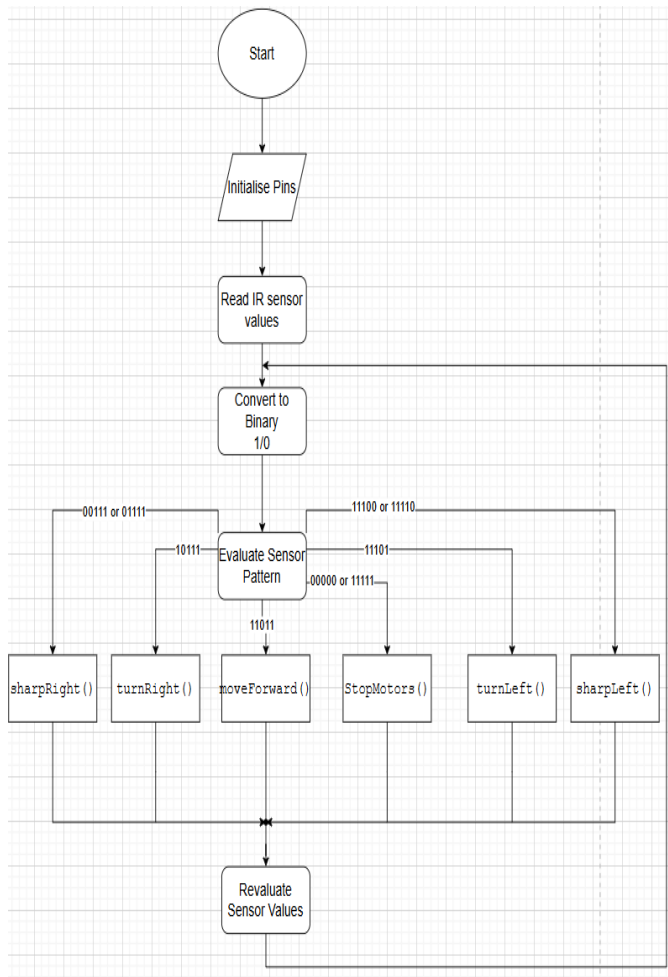
- Optimized: **$O(1)$ per control loop**

These functions allow quick, deterministic motor responses based on sensor input. The rule-based system, paired with these discrete actions, provides better handling on curved paths and intersections, even with physical imbalances and non-ideal feedback.

The motor driver (L298N) is signaled to control direction via digital pins (IN1–IN4) and speed via PWM signals on ENA and ENB. This approach proves more reliable given the system's inherent latency, analog noise, and mechanical imbalances. Another important aspect is the absence of motor feedback systems. The 6V, 1000 RPM DC motors used in our robot do not include encoders, meaning that we cannot measure wheel speeds or apply speed correction—a feature often crucial for accurate PID control.

As highlighted previously, the lack of feedback mechanisms severely limits the practical performance of proportional-integral-derivative loops.

In this context, a rule-based strategy allows us to exploit predictable sensor patterns and direct motor mappings to navigate basic to moderately complex tracks with acceptable performance. Though limited in adaptability and refinement, this method aligns well with the simplicity of our electronic setup and meets the needs of the internal university competition for which the robot is developed.



[Figure 9(Final Flowchart of Code)]

The core of the robot's control logic revolves around interpreting the readings of five IR sensors connected to analog pins A0 through A4 on the Arduino Uno. Each sensor value is compared against a predefined threshold to determine whether it detects a black line (low reflectivity) or white surface (high reflectivity), and these comparisons are stored as binary values in an array. The logic is inverted such that 0 indicates the sensor is detecting the black line and 1 indicates the white background. The control logic then evaluates various sensor combinations to infer the position of the line relative to the robot.

The motor actions are controlled via the L298N driver by sending HIGH/LOW signals to the directional pins (IN1–IN4) and modulating speed using PWM signals through the enable pins (ENA and ENB). This setup allows real-time decisions and dynamic adjustments to motor direction and speed. The use of simple digital logic rather than floating-point PID calculations makes the system lightweight and well-suited for the Arduino's limited processing capability.

VI. TRACK DESIGN

Track design is a fundamental aspect of evaluating the performance and capabilities of line follower robots. Effective track layouts typically incorporate a variety of elements such as straight paths, curves, sharp turns, and intersections to challenge the robot's navigation algorithms and sensor responsiveness. These features test the robot's ability to maintain line detection and execute precise maneuvers under different conditions [10].

For our specific application, we employ a track constructed using two parallel strips of black electrical tape placed on a white surface, resulting in a path width of 3.82 centimeters. This configuration provides a consistent and high-contrast line for the robot's infrared (IR) sensors to detect, facilitating accurate line tracking. The dual-strip design enhances the robot's ability to distinguish the path from the background, which is crucial for maintaining alignment and stability during operation [11]. Incorporating various track features such as curves and intersections not only tests the robot's mechanical design and sensor placement but also evaluates the effectiveness of its control algorithms. By systematically analyzing the robot's performance across these different track elements, we can identify areas for improvement and optimize the overall system for better reliability and efficiency.

Taking all into consideration, to evaluate the bot the final track below has been developed to assess its speed, accuracy and stability across the different features of it.



[Figure 10 (Finalised Track Layout)]

Curved Path (Top Section):

The upper portion of the track contains a long, winding curve. This section evaluates the robot's ability to smoothly follow a continuous path without oscillation. It requires precise adjustments to motor speeds based on sensor readings, testing the robot's real-time correction capability during gradual turns.

Four-Way Intersection (Left-Center):

This segment simulates a decision point where the robot could potentially move forward, turn left, or turn right. It tests the robot's rule-based logic to maintain its primary direction or select a path based on pre-programmed priority rules.

Straight Segment with Obstacle (Bottom-Left):

A short straight segment is partially blocked by a white physical object. While the robot shown does not use obstacle sensors, this placement introduces a manual challenge for alignment recovery and demonstrates how physical interference can affect line tracking performance.

T-Junction (Center):

The center of the track features a T-junction where the robot must choose between continuing forward or turning. This junction tests whether the robot can identify and act on multiple line inputs correctly using sensor pattern recognition.

Square Loop (Right-Center):

A clearly defined rectangular loop branches off the main track. This section tests the robot's ability to handle closed-loop navigation and its capacity to rejoin the main path without deviation. It also evaluates logic consistency when multiple valid paths are detected.

Crossed Paths (Bottom-Left to Top-Right):

Two track lines intersect perpendicularly, forming a crossroad. This challenges the robot to stay on its designated path without being misled by the intersecting line. It is a crucial test of line-following priority when sensor inputs are ambiguous.

VII. SCENARIO TESTING

To verify the effectiveness of the rule-based logic in real-world situations, the robot was tested on a variety of track scenarios, including straight paths, curves, sharp turns, junctions, and broken lines on the path. The robot's ability to follow the path, make decisions at intersections, and recover from line loss was evaluated. This validated the robustness and accuracy of controlling logic under practical conditions.

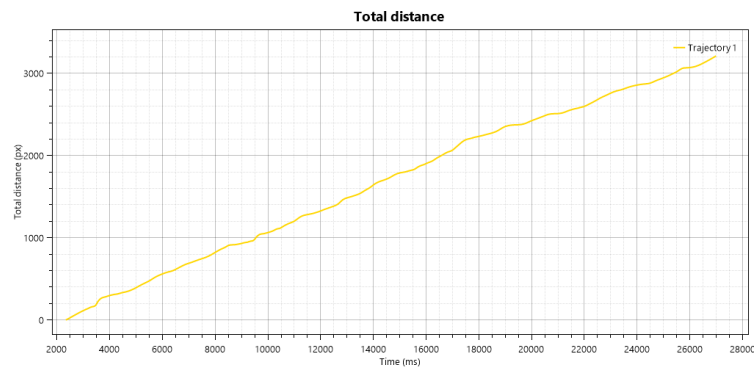
Test Segment	Four-Way Intersection – Left-Center
Objective	To test the robot's junction detection and rule-based path selection
Initial Conditions	Robot approaches the intersection head-on - Battery: Normal - IR sensor calibrated - Ambient light: Consistent
Expected Behaviour	Robot should detect the intersection and follow the predefined rule (e.g., go straight or turn right).
Sensor Input Pattern	All sensors detect black simultaneously at the center of the intersection
Control Logic Triggered	Detects multiple black sensors, stops momentum, carries it forward and chooses preferred direction (e.g., straight).
Actual Outcome	Performed extremely well moved smoothly first few runs, however, did tires get hot and skidding occurred but still maintained consistency with the loop.
Issues Observed	Tire heating, bubbles in track due to tape not being applied evenly causing irregularities in movement.
Remarks/Improvements	N/A

Test Segment	Curved Path – Top Section
Objective	To evaluate the robot's ability to maintain accurate line tracking through a continuous curve.
Initial Conditions	- Robot aligned on the line - Battery: Normal - IR sensor calibrated - Ambient light: Consistent
Expected Behaviour	Robot should navigate the curve smoothly with continuous adjustments to motor speeds. No oscillations or deviations.
Sensor Input Pattern	Center sensor = black, slight alternating detection on left or right sensors as the robot turns
Control Logic Triggered	If C is black → go forward. If L is black → reduce left motor speed. If R is black → reduce right motor speed.
Actual Outcome	Massive oscillations and overshooting not extremely smooth,
Issues Observed	Speed of which the section was completed was much slower than expected, and needed to re-calibrate the turning speed to reduce oscillation.
Remarks/Improvements	Turn functions re tuned, ride height of sensor adjusted.

Test Segment	Straight Segment with Obstacle – Bottom-Left
Objective	To observe how physical disturbances affect the robot's line-tracking and recovery behaviour
Initial Conditions	- Robot starts before the obstacle - Battery: Normal - IR sensor calibrated - Obstacle partially covers the line
Expected Behaviour	Robot may temporarily lose the line but should attempt to reacquire it and continue tracking
Sensor Input Pattern	Sudden white input from all sensors or random black/white due to displacement
Control Logic Triggered	Keeps moving forward and finds and centers itself on the line on the obstacles.
Actual Outcome	Very smooth transition, extremely stable, high torque in wheels so smoothly move up ramp structure.
Issues Observed	No issues extremely stable
Remarks/Improvements	Turning up base forward speed could help smoothness

Test Segment	T-Junction – Center
Objective	To test the robot's ability to detect a T-junction and make a directional decision (e.g., turn right).
Initial Conditions	- Robot approaches junction from the base of the "T" - Battery: Low - IR sensor calibrated
Expected Behaviour	Robot should detect the T-junction and turn according to predefined rules.
Sensor Input Pattern	Center = black, Left and Right sensors = black briefly as side lines appear
Control Logic Triggered	Turns of power to motors momentum carries forward and next decision is made based on binary pattern
Actual Outcome	Moves straight in the direction it was initially facing has clear direction and doesn't oscillate back and forth.
Issues Observed	Battery being low caused extremely slow movement, however did still perform task with stability. On the third run only one motor was supplied with current due to battery being extremely low. Recharge required
Remarks/Improvements	Power efficiency was understood to be weak across scenario testing due to only 3 to 4 runs being performed before significant speed and stability loss.

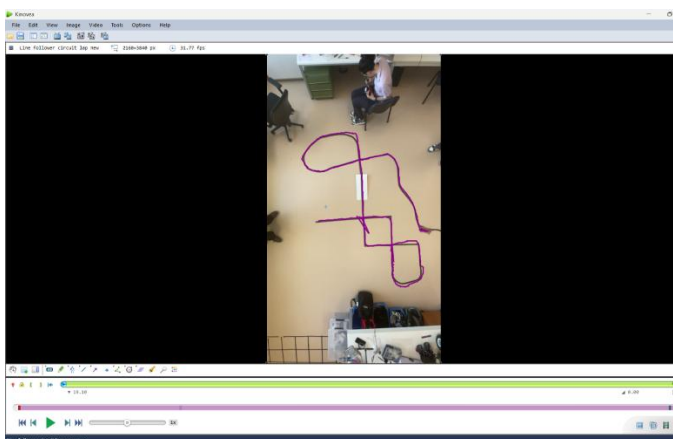
By recording the robot's position in each video frame, Kinovea generated a time series of x- and y-coordinates, which was then processed to calculate key kinematic variables. Total distance was derived from the cumulative frame-to-frame displacement, speed was computed by analysing the rate of positional change over time, and acceleration was determined as the rate of change of speed. These variables were plotted as Figures 11, 12, and 13, offering a time-based view of how the robot progressed through the track. The use of Kinovea provided a non-invasive, high-resolution method to analyse motion, particularly valuable for rule-based systems where discrete logic transitions can introduce abrupt or irregular kinematic behaviour. Its application has been validated in various biomechanics and engineering studies, demonstrating high reliability and accuracy in tracking movement using standard video footage [17]. This allowed for an effective evaluation of how well the robot's programmed logic handled diverse track features such as curves, intersections, and obstacles. The ability to link sudden changes in acceleration or velocity to specific decision events provided deep insight into the robot's behavioural consistency and control efficiency.



[Figure 12(Total distance covered over lap)]

VIII.LINE FOLLOWER PERFORMANCE DATA ANALYSIS

To evaluate the performance of the rule-based control system on a complex test track, it was essential to quantify the robot's kinematic behavior—its movement, velocity changes, and corrective actions. For this purpose, the open-source motion analysis software *Kinovea* was employed. Kinovea enables detailed video-based tracking by analyzing motion frame by frame. In this investigation, the software was used to track a fixed point on the robot as it completed a full lap around the custom-built test course.

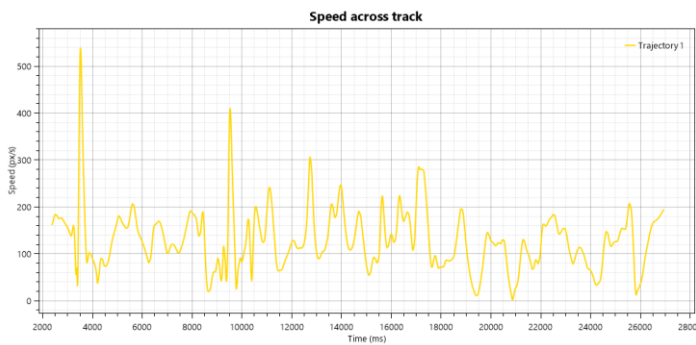


[Figure 11 (Kinovea Tracking of lap)]

The Total Distance graph shows a generally linear progression, indicating continuous forward motion with minimal interruption. This pattern implies that the robot maintained its trajectory along the intended line with limited deviations.

- **Linear Slope:**
 - The steady slope across much of the graph reflects successful tuning of the PID controller to sustain forward motion, particularly on straight and moderately curved segments.
- **Plateaus and Inflection Points:**
 - Flattening and slope reductions—particularly around 4,000 ms, 10,000 ms, and 26,000 ms—suggest hesitation or slowdown events, likely due to intersections, sharp curves, or physical disturbances. These areas highlight moments where the robot either briefly lost the line or required time to make corrective decisions.

The robot's ability to resume a linear trajectory following such events points to effective path recovery behaviour. The total distance data thus reflects both the base-level consistency of the system and its capacity to navigate interruptions in tracking.

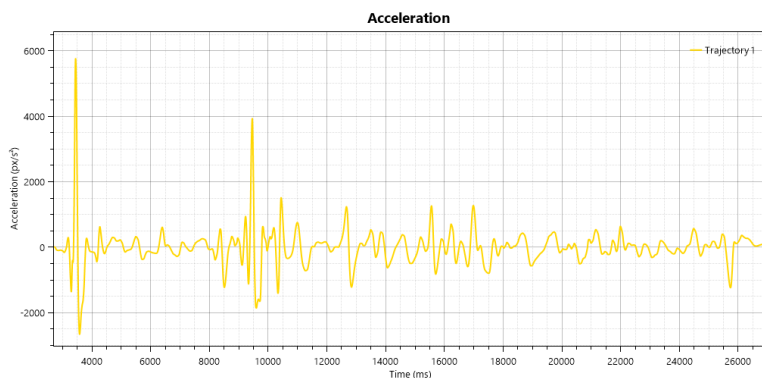


[Figure 13(Speed vs Time over the lap)]

The Speed graph reveals the dynamic nature of the robot's movement over time, with key fluctuations providing insight into control stability and responsiveness.

- **Baseline Operation:**
 - During stable periods, the robot maintained a velocity between 100–200 px/s. This range is indicative of smooth tracking performance and a well-balanced trade-off between speed and control.
- **Speed Dips and Spikes:**
 - Abrupt changes at times such as 3,500 ms, 9,500 ms, and 25,000 ms suggest reactive behaviour to challenging scenarios. For instance, the sudden drop and rise in speed around 3,500 ms could indicate the robot encountering a sharp turn or obstacle, requiring fast deceleration and reacceleration.

These speed transitions confirm the system's agility in adjusting to changing track geometries. While such fluctuations are expected in environments with high variability, smoother transitions could further improve lap efficiency and energy usage.



[Figure 14(The acceleration over time during Lap)]

The Acceleration graph provides a more granular view of the robot's control responsiveness, highlighting how rapidly it changes its velocity to stay aligned with the line.

- **Oscillatory Patterns:**
 - High-frequency accelerations and decelerations throughout the lap indicate continuous micro-adjustments to maintain alignment. This is

typical in PID-driven systems where the derivative term introduces sensitivity to error changes, especially in curves or uneven lighting.

- **High-Magnitude Corrections:**
 - Prominent acceleration spikes at ~3,500 ms, ~9,500 ms, and ~25,000 ms correspond with speed anomalies, further validating that these are moments of significant control effort—often at intersections or unexpected path deviations.

While responsive correction is essential, excessive or repeated high-magnitude acceleration may suggest overly aggressive control tuning. Smoother acceleration curves could be achieved through better filtering or gain optimization, enhancing both efficiency and motor lifespan.

Overall, the kinematic data reflects a robot that performs reliably in standard path-following conditions, with smooth distance progression and consistent speed through straight and gently curved segments. The rule-based logic demonstrates effective responsiveness to track features, as shown by speed adjustments and acceleration spikes aligned with junctions and turns. However, the presence of sharp acceleration fluctuations and brief deceleration zones suggests weaknesses in handling ambiguous inputs and recovering from misalignment. These findings highlight the system's strengths in simplicity and decisiveness, but also its sensitivity to sensor limitations and lack of smoothing in transitional behaviour.

IX. DISCUSSION AND LIMITATIONS

While the line-following robot demonstrated consistent performance under standard track conditions, several edge cases revealed limitations in both hardware and control logic. For instance, sharp turns and intersections caused temporary loss of line detection, resulting in speed dips and corrective acceleration spikes. These were evident in the performance graphs around timestamps like 3,500 ms and 25,000 ms, indicating the system's delayed response when handling abrupt path transitions.

A major limitation stemmed from the use of a rule-based control system without adaptive behaviour. Although effective for clear, high-contrast lines and predictable curves, this approach struggled in ambiguous situations—such as faded track lines, intersections with uneven lighting, or sudden changes in curvature. The reliance on binary thresholding in the IR sensor readings made the robot sensitive to noise, particularly in poorly lit environments or reflective surfaces.

The current setup also lacked filtering or smoothing techniques in speed and acceleration control. This led to jerky transitions during corrections, potentially stressing motors and reducing long-term efficiency. Improvements such as implementing a tuned PID controller (with gain optimization and filtering) or integrating more advanced logic like fuzzy or neural-based control could address these challenges.

Moreover, the mechanical design could benefit from additional stabilization elements—like caster wheels or a more rigid chassis—to reduce vibrations and improve line sensor consistency. Modular wiring and better cable management would also enhance ease of maintenance and system reliability.

X. CONCLUSION AND FUTURE WORK

This project successfully designed and built a functional line-following robot using a rule-based control system, basic IR sensing, and an Arduino-controlled motor driver. The robot reliably completed laps on a custom-built track and demonstrated the ability to recover from minor deviations, as verified by motion analysis with Kinovea.

Key achievements include:

- A stable hardware platform integrating sensors, motors, and logic control.
- A working rule-based logic system capable of interpreting 5-sensor inputs.
- Video-based kinematic evaluation for in-depth performance analysis.

However, several challenges emerged, particularly with handling complex paths and sudden line changes. The robot's responses in these cases were often abrupt and reactive, rather than anticipatory or adaptive. This highlighted the limitations of the rule-based approach when operating in dynamic or noisy environments. Lessons learned include the importance of tuning control parameters, considering environmental variability in design, and validating system behaviour using real-world data. Future development will focus on:

- Replacing the rule-based logic with PID or fuzzy logic control for smoother performance.
- Enhancing sensor processing through filtering or adaptive thresholds.
- Improving chassis stability and modularity for easier troubleshooting
- Introducing onboard feedback mechanisms (e.g., encoders or IMUs) for more precise control.

the project laid a strong foundation for a capable line-following robot and demonstrated how fundamental control concepts translate into real-world motion behaviour. With targeted refinements, the robot can evolve into a more robust and intelligent autonomous system.

REFERENCES

1. Haowen Jiang (2024). *Overview and Development of PID Control*. ResearchGate. https://www.researchgate.net/publication/380983965_Overview_and_development_of_PID_control
2. Islam, M. A Rahman (2016). *Design and Fabrication of Line Follower Robot*. Academia.edu. https://www.academia.edu/28545738/Design_and_Fabrication_of_Line_Follower_Robot
3. V. M. Alfaro et al. (2022). *A Review on PID Control and Its Applications*. *Frontiers in Control Engineering*, 3, Article 1061830. <https://www.frontiersin.org/articles/10.3389/fcteg.2022.1061830/>
4. Ahmed A. (2018). *Design and Implementation of a Line Follower Robot Based on Fuzzy Logic Controller by Using Microcontroller*. ResearchGate. <https://www.researchgate.net/publication/328891369>
5. Aryan R, Vishal D. et al. (2023). *Line Follower Robot using Arduino Uno and L298N Motor Driver*. *IRJET*, 10(7), 956–960. <https://www.irjet.net/archives/V10/I7/IRJET-V10I7124.pdf>
6. Weibin K., Haonan Z., Xiaofang Y. et al. (2024). *Application of PID Control in Voltage Regulation Systems*. *Scientific Reports*, 14, Article 79653. <https://www.nature.com/articles/s41598-024-79653-z>
7. Onur E Abdullah G. Burhan D. et al. (2024). *Line Follower Robot with PID Control*. ResearchGate. https://www.researchgate.net/publication/377666058_Line_Follower_Robot_with_PID_Control
8. Islam, A., & Alam, M. (2024). *PID Based Line Follower Robot*. ResearchGate. https://www.researchgate.net/publication/378545759_Line_Follower_Robot_with_PID_Control
9. (2024). *Design and Implementation of Autonomous Line Follower Robot*. KTH Royal Institute of Technology <https://www.divaportal.org/smash/get/diva2:1908071/FULLTEXT01.pdf>
10. Adnan R., Erik H. et al. (2017) *Design and Implementation of PID Control-based FSM Algorithm on Line Following Robot*, ResearchGate. https://www.researchgate.net/publication/326268217_Design_and_Implementation_of_PID_Control-based_FSM_Algorithm_on_Line_Following_Robot
11. Harrison, P. (2008). *Line Follower Test Track*. *Micromouse Online*. <https://micromouseonline.com/2008/11/16/line-follower-test-track/>
12. Parallax Inc. (2022). *Printable Tracks for Line Following*. <https://www.parallax.com/package/printable-tracks-for-line-following/>
13. PID Controller Optimization for Low-cost Line Follower Robots. *arXiv preprint arXiv:2111.04149*, 2021. <https://arxiv.org/pdf/2111.04149>
14. Optimization of PID Control for High Speed Line Tracking Robots. *Procedia Computer Science*, vol. 60, pp. 516–525, 2015. <https://www.sciencedirect.com/science/article/pii/S1877050915038302>
15. Design of a High-Speed Line Following Robot. *Simulation Modelling Practice and Theory*, vol. 51, pp. 1–11, 2015. <https://www.sciencedirect.com/science/article/pii/S0045790615002190>
16. Puig-Diví, A., Escalona-Marfil, C., Padullés-Riu, J. M., Buscà, B., & Padullés-Chando, J. (2019). *Validity and reliability of the Kinovea program in obtaining angles and distances using coordinates in 2D motion analysis*. *PLOS ONE*, 14(6), e0216448. <https://doi.org/10.1371/journal.pone.0216448>
17. Pilar Fernández-González „Aikaterini Koutsou „Alicia Cuesta-Gómez . (2020). *Reliability of Kinovea® software and agreement with a three-dimensional motion analysis system for measuring the knee angle during a single-leg squat*. *Sensors*, 20(11), 3154. <https://doi.org/10.3390/s20113154>